

SME to walk through the following aspects related to the project created:

1. `src/main/java` - Folder with application code:

```
package com.cognizant.spring_learn;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import java.util.HashMap;
import java.util.Map;

@RestController
public class HelloController {

    // 1. A simple text endpoint
    @GetMapping("/welcome")
    public String welcome() {
        return "Welcome to your first Spring Boot application!";
    }

    // 2. An endpoint that automatically returns JSON data
    @GetMapping("/status")
    public Map<String, String> getStatus() {
        Map<String, String> status = new HashMap<>();
        status.put(key: "status", value: "Up and Running");
        status.put(key: "environment", value: "Development");
        return status;
    }
}
```

2. `src/main/resources` - Folder for application configuration:

static	10-07-2026 21:01	File folder	
templates	10-07-2026 21:01	File folder	
application	11-07-2026 02:32	Properties Source ...	1 KB

3. src/ test/ java – folder with code for testing:

```
1 package com.cognizant.spring_learn;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class SpringLearnApplicationTests {
8
9     @Test
10     void contextLoads() {
11     }
12
13 }
14
```

4. SpringLearnApplication.java - Walkthrough the main() method.

```
1 package com.cognizant.spring_learn;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.boot.SpringApplication;
6 import org.springframework.boot.autoconfigure.SpringBootApplication;
7
8 @SpringBootApplication
9 public class SpringLearnApplication {
10
11     private static final Logger LOGGER = LoggerFactory.getLogger(SpringLearnApplication.class);
12
13     public static void main(String[] args) {
14         SpringApplication.run(SpringLearnApplication.class, args);
15         LOGGER.info("Inside main method: SpringLearnApplication started successfully!");
16     }
17 }
```

5. Purpose of **@SpringBootApplication** annotation

Annotation	What it does
@SpringBootConfiguration	A specialized version of @Configuration. It marks this class as a source of bean definitions for the application context.
@EnableAutoConfiguration	The "magic" of Spring Boot. It tells the framework to scan your project's classpath dependencies and automatically configure sensible defaults. <i>Example: Since spring-boot-starter-web is on your classpath, it automatically configures an embedded Tomcat server and sets up Jackson for JSON parsing.</i>

Annotation	What it does
@ComponentScan	Instructs Spring to search the current package (com.cognizant.springlearn) and all of its sub-packages for managed components (such as @Component, @RestController, @Service, or @Repository) and register them as Beans in the container.

6. pom.xml Walkthrough

6.1 Configuration Defined in the XML File

- **<parent>**: Inheriting from spring-boot-starter-parent acts as your project's Bill of Materials (BOM). It manages version control for dozens of matching dependencies so you don't have to manually specify version tags for every library.
- **<dependencies>**:
 - spring-boot-starter-web: Pulls in core Spring MVC libraries and the embedded Tomcat engine.
 - spring-boot-devtools: Enables fast application restarts and live reload capacities during active development.
- **<build>**: Declares the spring-boot-maven-plugin. This plugin packages your entire compiled application alongside its embedded dependencies into a single, executable **"fat" JAR** file.

6.2 The Dependency Hierarchy Tree

While your pom.xml only explicitly names a couple of core starter dependencies, Maven relies on **Transitive Dependencies** (dependencies of dependencies).

To view the dependency tree in **VS Code**:

1. Look at your bottom-left sidebar panel and expand the **MAVEN** tab.
2. Expand **spring-learn** → **Dependencies**.
3. You will see the expanded tree structure showing exactly how declaring spring-boot-starter-web silently pulled in all necessary foundational jars:

spring-learn [jar]

└─ Dependencies

└─ spring-boot-starter-web

└─ spring-boot-starter

└─ spring-boot-starter-json

| └─ jackson-databind (Converts objects to JSON)

└─ spring-boot-starter-tomcat (Embedded Web Server)

└─ spring-web

└─ spring-webmvc (Handles routing & REST Controllers)